

Complete Implementation Commands

Track Feature Integration - Agent-Ready Command File

This document consolidates ALL implementation plans and code commands into a single agent-ready instruction file. An AI coding agent can execute these commands step-by-step to integrate 15 new track-specific features into the existing F1 2026 Predictor program. The file covers: 24 per-circuit JSON data files, a loader module, ML feature expansion (35 to 52 features), Monte Carlo circuit-specific parameters, self-improvement per-circuit bias corrections, and full validation procedures. Every command references exact file paths, function names, line numbers, and variable names from the existing codebase.

Quick Reference

Item	Current State	After Implementation
Track features	4 basic encodings (track_type, overtaking, power_unit, downforce)	15+ numeric features per circuit
Feature count	35 features (FEATURE_NAMES)	52 features (+15 track features + 2 derived)
Track data source	CIRCUITS dict in config.py (hard-coded strings)	Per-circuit JSON files + loader module
SC probability	Single SC_PROBABILITY dict in config.py	Per-circuit sc_prob in JSON (same source, richer context)
Self-improvement	3-category (street/hybrid/permanent) bias	Per-circuit bias corrections (24 individual circuits)
Monte Carlo	Generic weather/SC noise parameters	Circuit-specific variance multipliers
New files	0	24 JSON + 1 loader + 1 validation script
Modified files	0	config.py, predictor.py, monte_carlo.py, self_improvement.py, main_logic.py, data_fetcher.py, chip_advisor.py

PHASE 1: Create Track Feature Data Files

Create 24 individual JSON files in a new **track_features/** directory, one per circuit. Each file contains 15+ track-specific features that replace and extend the current 4 basic circuit encodings. The data comes from the compiled F1 2026 Track Features Database (XLSX).

1.1 Create Directory Structure

STEP 1: Create the track_features directory

```
mkdir -p track_features
touch track_features/__init__.py
```

The directory must be at the same level as config.py, predictor.py, etc. The __init__.py file makes it a proper Python package for imports.

1.2 JSON Schema

Every JSON file MUST follow this exact schema. The keys match the new feature names that will be added to FEATURE_NAMES in predictor.py. Missing keys will cause KeyError at runtime, so every field must be present.

```
{
  "circuit_key": "australia",
  "circuit_name": "Albert Park Circuit",
  "grand_prix": "Australian Grand Prix",
  "city": "Melbourne",
  "country": "AU",
  "circuit_type": "hybrid",
  "circuit_length_km": 5.278,
  "num_laps": 58,
  "num_turns": 14,
  "direction": "clockwise",
  "altitude_m": 10,
  "longest_straight_m": 800,
  "track_width_m": 12,
  "sm_zones": 3,
  "downforce_level": 4,
  "overtaking_difficulty": 3,
  "tire_degradation": 3,
  "sc_probability": 0.55,
  "vsc_probability": 0.40,
  "first_lap_incident_risk": 0.08,
  "quali_importance": 4,
  "weather_variability": 3,
  "pit_time_loss_s": 21.5,
  "deg_compound_delta": 0.3,
  "avg_speed_kmh": 225,
  "elevation_change_m": 0,
  "overtake_mode_efficiency": 0.7
}
```

1.3 Create All 24 JSON Files

Create each file as **track_features/{circuit_key}.json**. Below is the complete data for every circuit. The agent MUST create all 24 files with this exact data. Do NOT calculate or estimate - use these precise values from the research database.

STEP : Create track_features/australia.json

```
{
  "circuit_key": "australia",
  "circuit_name": "Albert Park Circuit",
  "grand_prix": "Australian Grand Prix",
  "city": "Melbourne",
  "country": "AU",
  "circuit_type": "hybrid",
  "circuit_length_km": 5.278,
  "num_laps": 58,
  "num_turns": 14,
  "direction": "clockwise",
```

```

"altitude_m": 10,
"longest_straight_m": 800,
"track_width_m": 12,
"sm_zones": 3,
"downforce_level": 4,
"overtaking_difficulty": 3,
"tire_degradation": 3,
"sc_probability": 0.55,
"vsc_probability": 0.4,
"first_lap_incident_risk": 0.08,
"quali_importance": 4,
"weather_variability": 3,
"pit_time_loss_s": 21.5,
"deg_compound_delta": 0.3,
"avg_speed_kmh": 225,
"elevation_change_m": 0,
"overtake_mode_efficiency": 0.7
}

```

STEP : Create track_features/china.json

```

{
  "circuit_key": "china",
  "circuit_name": "Shanghai International Circuit",
  "grand_prix": "Chinese Grand Prix",
  "city": "Shanghai",
  "country": "CN",
  "circuit_type": "permanent",
  "circuit_length_km": 5.451,
  "num_laps": 56,
  "num_turns": 16,
  "direction": "clockwise",
  "altitude_m": 5,
  "longest_straight_m": 1170,
  "track_width_m": 14,
  "sm_zones": 2,
  "downforce_level": 3,
  "overtaking_difficulty": 2,
  "tire_degradation": 3,
  "sc_probability": 0.4,
  "vsc_probability": 0.3,
  "first_lap_incident_risk": 0.06,
  "quali_importance": 3,
  "weather_variability": 2,
  "pit_time_loss_s": 22.0,
  "deg_compound_delta": 0.2,
  "avg_speed_kmh": 220,
  "elevation_change_m": 5,
  "overtake_mode_efficiency": 0.65
}

```

STEP : Create track_features/japan.json

```

{
  "circuit_key": "japan",
  "circuit_name": "Suzuka International Racing Course",
  "grand_prix": "Japanese Grand Prix",
  "city": "Suzuka",
  "country": "JP",
  "circuit_type": "permanent",
  "circuit_length_km": 5.807,
  "num_laps": 53,
  "num_turns": 18,
  "direction": "figure-8",
  "altitude_m": 45,
  "longest_straight_m": 1000,
  "track_width_m": 12,
  "sm_zones": 2,
  "downforce_level": 5,
  "overtaking_difficulty": 4,
  "tire_degradation": 3,
  "sc_probability": 0.35,
}

```

```
"vsc_probability": 0.25,  
"first_lap_incident_risk": 0.07,  
"quali_importance": 5,  
"weather_variability": 4,  
"pit_time_loss_s": 22.5,  
"deg_compound_delta": 0.4,  
"avg_speed_kmh": 230,  
"elevation_change_m": 40,  
"overtake_mode_efficiency": 0.5  
}
```

STEP : Create track_features/miami.json

```
{  
  "circuit_key": "miami",  
  "circuit_name": "Miami International Autodrome",  
  "grand_prix": "Miami Grand Prix",  
  "city": "Miami",  
  "country": "US",  
  "circuit_type": "street",  
  "circuit_length_km": 5.412,  
  "num_laps": 57,  
  "num_turns": 19,  
  "direction": "counter-clockwise",  
  "altitude_m": 0,  
  "longest_straight_m": 1150,  
  "track_width_m": 11,  
  "sm_zones": 3,  
  "downforce_level": 3,  
  "overtaking_difficulty": 2,  
  "tire_degradation": 3,  
  "sc_probability": 0.45,  
  "vsc_probability": 0.35,  
  "first_lap_incident_risk": 0.07,  
  "quali_importance": 3,  
  "weather_variability": 3,  
  "pit_time_loss_s": 23.0,  
  "deg_compound_delta": 0.3,  
  "avg_speed_kmh": 215,  
  "elevation_change_m": 0,  
  "overtake_mode_efficiency": 0.7  
}
```

STEP : Create track_features/canada.json

```
{  
  "circuit_key": "canada",  
  "circuit_name": "Circuit Gilles Villeneuve",  
  "grand_prix": "Canadian Grand Prix",  
  "city": "Montreal",  
  "country": "CA",  
  "circuit_type": "hybrid",  
  "circuit_length_km": 4.361,  
  "num_laps": 70,  
  "num_turns": 14,  
  "direction": "clockwise",  
  "altitude_m": 10,  
  "longest_straight_m": 1050,  
  "track_width_m": 13,  
  "sm_zones": 3,  
  "downforce_level": 2,  
  "overtaking_difficulty": 1,  
  "tire_degradation": 3,  
  "sc_probability": 0.55,  
  "vsc_probability": 0.4,  
  "first_lap_incident_risk": 0.09,  
  "quali_importance": 3,  
  "weather_variability": 3,  
  "pit_time_loss_s": 20.0,  
  "deg_compound_delta": 0.2,  
  "avg_speed_kmh": 215,  
  "elevation_change_m": 5,  
}
```

```
"overtake_mode_efficiency": 0.75
}
```

STEP : Create track_features/monaco.json

```
{
  "circuit_key": "monaco",
  "circuit_name": "Circuit de Monaco",
  "grand_prix": "Monaco Grand Prix",
  "city": "Monte Carlo",
  "country": "MC",
  "circuit_type": "street",
  "circuit_length_km": 3.337,
  "num_laps": 78,
  "num_turns": 19,
  "direction": "clockwise",
  "altitude_m": 25,
  "longest_straight_m": 500,
  "track_width_m": 10,
  "sm_zones": 1,
  "downforce_level": 5,
  "overtaking_difficulty": 5,
  "tire_degradation": 2,
  "sc_probability": 0.7,
  "vsc_probability": 0.55,
  "first_lap_incident_risk": 0.06,
  "quali_importance": 5,
  "weather_variability": 2,
  "pit_time_loss_s": 24.0,
  "deg_compound_delta": 0.5,
  "avg_speed_kmh": 160,
  "elevation_change_m": 45,
  "overtake_mode_efficiency": 0.15
}
```

STEP : Create track_features/spain.json

```
{
  "circuit_key": "spain",
  "circuit_name": "Circuit de Barcelona-Catalunya",
  "grand_prix": "Barcelona Grand Prix",
  "city": "Barcelona",
  "country": "ES",
  "circuit_type": "permanent",
  "circuit_length_km": 4.657,
  "num_laps": 66,
  "num_turns": 16,
  "direction": "clockwise",
  "altitude_m": 120,
  "longest_straight_m": 1040,
  "track_width_m": 12,
  "sm_zones": 2,
  "downforce_level": 3,
  "overtaking_difficulty": 2,
  "tire_degradation": 4,
  "sc_probability": 0.25,
  "vsc_probability": 0.2,
  "first_lap_incident_risk": 0.05,
  "quali_importance": 3,
  "weather_variability": 2,
  "pit_time_loss_s": 22.0,
  "deg_compound_delta": 0.4,
  "avg_speed_kmh": 215,
  "elevation_change_m": 40,
  "overtake_mode_efficiency": 0.5
}
```

STEP : Create track_features/austria.json

```
{
  "circuit_key": "austria",
```

```

"circuit_name": "Red Bull Ring",
"grand_prix": "Austrian Grand Prix",
"city": "Spielberg",
"country": "AT",
"circuit_type": "permanent",
"circuit_length_km": 4.318,
"num_laps": 71,
"num_turns": 10,
"direction": "clockwise",
"altitude_m": 720,
"longest_straight_m": 800,
"track_width_m": 13,
"sm_zones": 3,
"downforce_level": 3,
"overtaking_difficulty": 2,
"tire_degradation": 2,
"sc_probability": 0.35,
"vsc_probability": 0.25,
"first_lap_incident_risk": 0.06,
"quali_importance": 2,
"weather_variability": 3,
"pit_time_loss_s": 20.5,
"deg_compound_delta": 0.2,
"avg_speed_kmh": 235,
"elevation_change_m": 60,
"overtake_mode_efficiency": 0.75
}

```

STEP : Create track_features/britain.json

```

{
  "circuit_key": "britain",
  "circuit_name": "Silverstone Circuit",
  "grand_prix": "British Grand Prix",
  "city": "Silverstone",
  "country": "GB",
  "circuit_type": "permanent",
  "circuit_length_km": 5.891,
  "num_laps": 52,
  "num_turns": 18,
  "direction": "clockwise",
  "altitude_m": 140,
  "longest_straight_m": 1050,
  "track_width_m": 14,
  "sm_zones": 3,
  "downforce_level": 4,
  "overtaking_difficulty": 2,
  "tire_degradation": 3,
  "sc_probability": 0.4,
  "vsc_probability": 0.35,
  "first_lap_incident_risk": 0.07,
  "quali_importance": 3,
  "weather_variability": 4,
  "pit_time_loss_s": 22.5,
  "deg_compound_delta": 0.3,
  "avg_speed_kmh": 235,
  "elevation_change_m": 25,
  "overtake_mode_efficiency": 0.65
}

```

STEP : Create track_features/belgium.json

```

{
  "circuit_key": "belgium",
  "circuit_name": "Circuit de Spa-Francorchamps",
  "grand_prix": "Belgian Grand Prix",
  "city": "Spa-Francorchamps",
  "country": "BE",
  "circuit_type": "permanent",
  "circuit_length_km": 7.004,
  "num_laps": 44,
  "num_turns": 20,

```

```

"direction": "clockwise",
"altitude_m": 400,
"longest_straight_m": 1150,
"track_width_m": 14,
"sm_zones": 3,
"downforce_level": 2,
"overtaking_difficulty": 1,
"tire_degradation": 3,
"sc_probability": 0.45,
"vsc_probability": 0.4,
"first_lap_incident_risk": 0.08,
"quali_importance": 3,
"weather_variability": 5,
"pit_time_loss_s": 21.0,
"deg_compound_delta": 0.2,
"avg_speed_kmh": 230,
"elevation_change_m": 100,
"overtake_mode_efficiency": 0.7
}

```

STEP : Create track_features/hungary.json

```

{
  "circuit_key": "hungary",
  "circuit_name": "Hungaroring",
  "grand_prix": "Hungarian Grand Prix",
  "city": "Budapest",
  "country": "HU",
  "circuit_type": "permanent",
  "circuit_length_km": 4.381,
  "num_laps": 70,
  "num_turns": 14,
  "direction": "clockwise",
  "altitude_m": 20,
  "longest_straight_m": 600,
  "track_width_m": 12,
  "sm_zones": 1,
  "downforce_level": 5,
  "overtaking_difficulty": 4,
  "tire_degradation": 4,
  "sc_probability": 0.3,
  "vsc_probability": 0.25,
  "first_lap_incident_risk": 0.05,
  "quali_importance": 5,
  "weather_variability": 2,
  "pit_time_loss_s": 22.0,
  "deg_compound_delta": 0.5,
  "avg_speed_kmh": 195,
  "elevation_change_m": 30,
  "overtake_mode_efficiency": 0.2
}

```

STEP : Create track_features/netherlands.json

```

{
  "circuit_key": "netherlands",
  "circuit_name": "Circuit Zandvoort",
  "grand_prix": "Dutch Grand Prix",
  "city": "Zandvoort",
  "country": "NL",
  "circuit_type": "permanent",
  "circuit_length_km": 4.259,
  "num_laps": 72,
  "num_turns": 14,
  "direction": "clockwise",
  "altitude_m": 5,
  "longest_straight_m": 650,
  "track_width_m": 11,
  "sm_zones": 1,
  "downforce_level": 5,
  "overtaking_difficulty": 5,
  "tire_degradation": 3,

```

```
"sc_probability": 0.35,
"vsc_probability": 0.25,
"first_lap_incident_risk": 0.07,
"quali_importance": 5,
"weather_variability": 3,
"pit_time_loss_s": 22.5,
"deg_compound_delta": 0.4,
"avg_speed_kmh": 205,
"elevation_change_m": 15,
"overtake_mode_efficiency": 0.2
}
```

STEP : Create track_features/italy.json

```
{
  "circuit_key": "italy",
  "circuit_name": "Autodromo Nazionale Monza",
  "grand_prix": "Italian Grand Prix",
  "city": "Monza",
  "country": "IT",
  "circuit_type": "permanent",
  "circuit_length_km": 5.793,
  "num_laps": 53,
  "num_turns": 11,
  "direction": "clockwise",
  "altitude_m": 155,
  "longest_straight_m": 1320,
  "track_width_m": 12,
  "sm_zones": 4,
  "downforce_level": 1,
  "overtaking_difficulty": 1,
  "tire_degradation": 2,
  "sc_probability": 0.4,
  "vsc_probability": 0.35,
  "first_lap_incident_risk": 0.08,
  "quali_importance": 2,
  "weather_variability": 2,
  "pit_time_loss_s": 20.0,
  "deg_compound_delta": 0.1,
  "avg_speed_kmh": 250,
  "elevation_change_m": 25,
  "overtake_mode_efficiency": 0.85
}
```

STEP : Create track_features/madring.json

```
{
  "circuit_key": "madring",
  "circuit_name": "Madring Circuit",
  "grand_prix": "Spanish Grand Prix",
  "city": "Madrid",
  "country": "ES",
  "circuit_type": "street",
  "circuit_length_km": 5.474,
  "num_laps": 55,
  "num_turns": 17,
  "direction": "counter-clockwise",
  "altitude_m": 15,
  "longest_straight_m": 900,
  "track_width_m": 12,
  "sm_zones": 2,
  "downforce_level": 3,
  "overtaking_difficulty": 3,
  "tire_degradation": 3,
  "sc_probability": 0.45,
  "vsc_probability": 0.35,
  "first_lap_incident_risk": 0.08,
  "quali_importance": 4,
  "weather_variability": 3,
  "pit_time_loss_s": 22.0,
  "deg_compound_delta": 0.3,
  "avg_speed_kmh": 215,
}
```

```
"elevation_change_m": 10,  
"overtake_mode_efficiency": 0.6  
}
```

STEP : Create track_features/azerbaijan.json

```
{  
  "circuit_key": "azerbaijan",  
  "circuit_name": "Baku City Circuit",  
  "grand_prix": "Azerbaijan Grand Prix",  
  "city": "Baku",  
  "country": "AZ",  
  "circuit_type": "street",  
  "circuit_length_km": 6.003,  
  "num_laps": 51,  
  "num_turns": 20,  
  "direction": "counter-clockwise",  
  "altitude_m": 0,  
  "longest_straight_m": 2200,  
  "track_width_m": 13,  
  "sm_zones": 3,  
  "downforce_level": 1,  
  "overtaking_difficulty": 1,  
  "tire_degradation": 2,  
  "sc_probability": 0.65,  
  "vsc_probability": 0.5,  
  "first_lap_incident_risk": 0.09,  
  "quali_importance": 2,  
  "weather_variability": 3,  
  "pit_time_loss_s": 21.0,  
  "deg_compound_delta": 0.1,  
  "avg_speed_kmh": 220,  
  "elevation_change_m": 0,  
  "overtake_mode_efficiency": 0.85  
}
```

STEP : Create track_features/singapore.json

```
{  
  "circuit_key": "singapore",  
  "circuit_name": "Marina Bay Street Circuit",  
  "grand_prix": "Singapore Grand Prix",  
  "city": "Singapore",  
  "country": "SG",  
  "circuit_type": "street",  
  "circuit_length_km": 4.94,  
  "num_laps": 62,  
  "num_turns": 23,  
  "direction": "counter-clockwise",  
  "altitude_m": 5,  
  "longest_straight_m": 800,  
  "track_width_m": 10,  
  "sm_zones": 1,  
  "downforce_level": 5,  
  "overtaking_difficulty": 5,  
  "tire_degradation": 4,  
  "sc_probability": 0.75,  
  "vsc_probability": 0.55,  
  "first_lap_incident_risk": 0.08,  
  "quali_importance": 5,  
  "weather_variability": 3,  
  "pit_time_loss_s": 24.5,  
  "deg_compound_delta": 0.5,  
  "avg_speed_kmh": 175,  
  "elevation_change_m": 10,  
  "overtake_mode_efficiency": 0.15  
}
```

STEP : Create track_features/usa.json

```
{
  "circuit_key": "usa",
  "circuit_name": "Circuit of the Americas",
  "grand_prix": "United States Grand Prix",
  "city": "Austin",
  "country": "US",
  "circuit_type": "permanent",
  "circuit_length_km": 5.513,
  "num_laps": 56,
  "num_turns": 20,
  "direction": "counter-clockwise",
  "altitude_m": 130,
  "longest_straight_m": 1000,
  "track_width_m": 14,
  "sm_zones": 3,
  "downforce_level": 3,
  "overtaking_difficulty": 2,
  "tire_degradation": 3,
  "sc_probability": 0.45,
  "vsc_probability": 0.35,
  "first_lap_incident_risk": 0.08,
  "quali_importance": 3,
  "weather_variability": 3,
  "pit_time_loss_s": 22.0,
  "deg_compound_delta": 0.3,
  "avg_speed_kmh": 220,
  "elevation_change_m": 40,
  "overtake_mode_efficiency": 0.6
}
```

STEP : Create track_features/mexico.json

```
{
  "circuit_key": "mexico",
  "circuit_name": "Autodromo Hermanos Rodriguez",
  "grand_prix": "Mexico City Grand Prix",
  "city": "Mexico City",
  "country": "MX",
  "circuit_type": "permanent",
  "circuit_length_km": 4.304,
  "num_laps": 71,
  "num_turns": 17,
  "direction": "clockwise",
  "altitude_m": 2240,
  "longest_straight_m": 1200,
  "track_width_m": 14,
  "sm_zones": 3,
  "downforce_level": 2,
  "overtaking_difficulty": 1,
  "tire_degradation": 2,
  "sc_probability": 0.4,
  "vsc_probability": 0.3,
  "first_lap_incident_risk": 0.08,
  "quali_importance": 3,
  "weather_variability": 4,
  "pit_time_loss_s": 20.5,
  "deg_compound_delta": 0.2,
  "avg_speed_kmh": 225,
  "elevation_change_m": 15,
  "overtake_mode_efficiency": 0.7
}
```

STEP : Create track_features/brazil.json

```
{
  "circuit_key": "brazil",
  "circuit_name": "Autodromo Jose Carlos Pace",
  "grand_prix": "Brazilian Grand Prix",
  "city": "Sao Paulo",
  "country": "BR",
  "circuit_type": "permanent",
  "circuit_length_km": 4.309,
  "num_laps": 71,
```

```

    "num_turns": 15,
    "direction": "counter-clockwise",
    "altitude_m": 780,
    "longest_straight_m": 900,
    "track_width_m": 13,
    "sm_zones": 2,
    "downforce_level": 3,
    "overtaking_difficulty": 2,
    "tire_degradation": 3,
    "sc_probability": 0.55,
    "vsc_probability": 0.45,
    "first_lap_incident_risk": 0.09,
    "quali_importance": 3,
    "weather_variability": 5,
    "pit_time_loss_s": 21.5,
    "deg_compound_delta": 0.3,
    "avg_speed_kmh": 215,
    "elevation_change_m": 60,
    "overtake_mode_efficiency": 0.6
  }
}

```

STEP : Create track_features/las_vegas.json

```

{
  "circuit_key": "las_vegas",
  "circuit_name": "Las Vegas Strip Circuit",
  "grand_prix": "Las Vegas Grand Prix",
  "city": "Las Vegas",
  "country": "US",
  "circuit_type": "street",
  "circuit_length_km": 6.201,
  "num_laps": 50,
  "num_turns": 17,
  "direction": "counter-clockwise",
  "altitude_m": 10,
  "longest_straight_m": 1950,
  "track_width_m": 14,
  "sm_zones": 3,
  "downforce_level": 1,
  "overtaking_difficulty": 1,
  "tire_degradation": 2,
  "sc_probability": 0.5,
  "vsc_probability": 0.35,
  "first_lap_incident_risk": 0.07,
  "quali_importance": 2,
  "weather_variability": 2,
  "pit_time_loss_s": 21.0,
  "deg_compound_delta": 0.1,
  "avg_speed_kmh": 235,
  "elevation_change_m": 0,
  "overtake_mode_efficiency": 0.8
}

```

STEP : Create track_features/qatar.json

```

{
  "circuit_key": "qatar",
  "circuit_name": "Lusail International Circuit",
  "grand_prix": "Qatar Grand Prix",
  "city": "Lusail",
  "country": "QA",
  "circuit_type": "permanent",
  "circuit_length_km": 5.38,
  "num_laps": 57,
  "num_turns": 16,
  "direction": "clockwise",
  "altitude_m": 8,
  "longest_straight_m": 1050,
  "track_width_m": 12,
  "sm_zones": 2,
  "downforce_level": 3,
  "overtaking_difficulty": 2,
}

```

```
"tire_degradation": 5,
"sc_probability": 0.3,
"vsc_probability": 0.25,
"first_lap_incident_risk": 0.05,
"quali_importance": 3,
"weather_variability": 3,
"pit_time_loss_s": 23.0,
"deg_compound_delta": 0.4,
"avg_speed_kmh": 220,
"elevation_change_m": 5,
"overtake_mode_efficiency": 0.55
}
```

STEP : Create track_features/abu_dhabi.json

```
{
  "circuit_key": "abu_dhabi",
  "circuit_name": "Yas Marina Circuit",
  "grand_prix": "Abu Dhabi Grand Prix",
  "city": "Abu Dhabi",
  "country": "AE",
  "circuit_type": "permanent",
  "circuit_length_km": 5.281,
  "num_laps": 58,
  "num_turns": 16,
  "direction": "counter-clockwise",
  "altitude_m": 5,
  "longest_straight_m": 1140,
  "track_width_m": 14,
  "sm_zones": 2,
  "downforce_level": 3,
  "overtaking_difficulty": 2,
  "tire_degradation": 3,
  "sc_probability": 0.3,
  "vsc_probability": 0.25,
  "first_lap_incident_risk": 0.05,
  "quali_importance": 3,
  "weather_variability": 2,
  "pit_time_loss_s": 22.5,
  "deg_compound_delta": 0.3,
  "avg_speed_kmh": 220,
  "elevation_change_m": 10,
  "overtake_mode_efficiency": 0.55
}
```

STEP : Create track_features/bahrain.json

```
{
  "circuit_key": "bahrain",
  "circuit_name": "Bahrain International Circuit",
  "grand_prix": "Bahrain Grand Prix",
  "city": "Sakhir",
  "country": "BH",
  "circuit_type": "permanent",
  "circuit_length_km": 5.412,
  "num_laps": 57,
  "num_turns": 15,
  "direction": "clockwise",
  "altitude_m": 10,
  "longest_straight_m": 1050,
  "track_width_m": 14,
  "sm_zones": 3,
  "downforce_level": 3,
  "overtaking_difficulty": 2,
  "tire_degradation": 4,
  "sc_probability": 0.35,
  "vsc_probability": 0.3,
  "first_lap_incident_risk": 0.06,
  "quali_importance": 3,
  "weather_variability": 3,
  "pit_time_loss_s": 22.0,
  "deg_compound_delta": 0.3,
}
```

```
"avg_speed_kmh": 215,  
"elevation_change_m": 5,  
"overtake_mode_efficiency": 0.65  
}
```

STEP : Create track_features/saudi_arabia.json

```
{  
  "circuit_key": "saudi_arabia",  
  "circuit_name": "Jeddah Corniche Circuit",  
  "grand_prix": "Saudi Arabian Grand Prix",  
  "city": "Jeddah",  
  "country": "SA",  
  "circuit_type": "street",  
  "circuit_length_km": 6.174,  
  "num_laps": 50,  
  "num_turns": 27,  
  "direction": "counter-clockwise",  
  "altitude_m": 0,  
  "longest_straight_m": 1500,  
  "track_width_m": 12,  
  "sm_zones": 3,  
  "downforce_level": 2,  
  "overtaking_difficulty": 1,  
  "tire_degradation": 2,  
  "sc_probability": 0.65,  
  "vsc_probability": 0.45,  
  "first_lap_incident_risk": 0.09,  
  "quali_importance": 3,  
  "weather_variability": 2,  
  "pit_time_loss_s": 22.0,  
  "deg_compound_delta": 0.2,  
  "avg_speed_kmh": 240,  
  "elevation_change_m": 0,  
  "overtake_mode_efficiency": 0.75  
}
```

PHASE 2: Create Track Features Loader Module

Create the **track_features_loader.py** module that loads, caches, and validates all per-circuit JSON files. This module provides a clean API for the rest of the program to access track features without knowing about the JSON file structure.

2.1 Create track_features_loader.py

Place this file at the project root (same level as config.py). It must be importable from predictor.py, monte_carlo.py, self_improvement.py, and chip_advisor.py.

```
"""
track_features_loader.py - Per-circuit track feature loader with caching and validation.
Loads JSON files from track_features/ directory, validates schema, provides lookup API.
"""

import json
from pathlib import Path
from typing import Optional, Dict, Any

_TRACK_DIR = Path(__file__).parent / "track_features"

# Required keys with expected types for validation
_REQUIRED_KEYS = {
    "circuit_key": str,
    "circuit_name": str,
    "grand_prix": str,
    "city": str,
    "country": str,
    "circuit_type": str,
    "circuit_length_km": (int, float),
    "num_laps": int,
    "num_turns": int,
    "direction": str,
    "altitude_m": (int, float),
    "longest_straight_m": (int, float),
    "track_width_m": (int, float),
    "sm_zones": int,
    "downforce_level": (int, float),
    "overtaking_difficulty": (int, float),
    "tire_degradation": (int, float),
    "sc_probability": (int, float),
    "vsc_probability": (int, float),
    "first_lap_incident_risk": (int, float),
    "quali_importance": (int, float),
    "weather_variability": (int, float),
    "pit_time_loss_s": (int, float),
    "deg_compound_delta": (int, float),
    "avg_speed_kmh": (int, float),
    "elevation_change_m": (int, float),
    "overtake_mode_efficiency": (int, float),
}

# In-memory cache: circuit_key -> dict
_cache: Dict[str, Dict[str, Any]] = {}
_validation_errors: list = []

def _validate(data: dict, filepath: str) -> bool:
    """Validate that a loaded JSON has all required keys with correct types."""
    errors = []
    for key, expected_type in _REQUIRED_KEYS.items():
        if key not in data:
            errors.append(f"Missing key: {key}")
        elif not isinstance(data[key], expected_type):
            errors.append(
                f"Key '{key}': expected {expected_type}, got {type(data[key]).__name__}"
            )
    # Range checks for critical numeric fields
    if "downforce_level" in data and not (1 <= data["downforce_level"] <= 5):
        errors.append("downforce_level must be 1-5")
    if "overtaking_difficulty" in data and not (1 <= data["overtaking_difficulty"] <= 5):
```

```

        errors.append("overtaking_difficulty must be 1-5")
    if "tire_degradation" in data and not (1 <= data["tire_degradation"] <= 5):
        errors.append("tire_degradation must be 1-5")
    if "sc_probability" in data and not (0 <= data["sc_probability"] <= 1):
        errors.append("sc_probability must be 0.0-1.0")
    if errors:
        _validation_errors.append(f"{filepath}: {'; '.join(errors)}")
    return False
return True

def load_track_features(circuit_key: str) -> Optional[Dict[str, Any]]:
    """Load track features for a given circuit_key. Returns None if not found."""
    if circuit_key in _cache:
        return _cache[circuit_key]

    filepath = _TRACK_DIR / f"{circuit_key}.json"
    if not filepath.exists():
        return None

    try:
        with open(filepath, "r", encoding="utf-8") as f:
            data = json.load(f)
    except (json.JSONDecodeError, OSError) as e:
        _validation_errors.append(f"{filepath}: {e}")
        return None

    if not _validate(data, str(filepath)):
        return None

    _cache[circuit_key] = data
    return data

def load_all_track_features() -> Dict[str, Dict[str, Any]]:
    """Load all track features from the track_features/ directory."""
    if _cache and len(_cache) >= 24:
        return dict(_cache)

    for json_file in sorted(_TRACK_DIR.glob("*.json")):
        circuit_key = json_file.stem
        if circuit_key not in _cache:
            load_track_features(circuit_key)

    return dict(_cache)

def get_track_feature(circuit_key: str, feature_name: str, default=None):
    """Get a single feature value for a circuit. Returns default if not found."""
    data = load_track_features(circuit_key)
    if data is None:
        return default
    return data.get(feature_name, default)

def get_validation_errors() -> list:
    """Return any validation errors from loading."""
    return list(_validation_errors)

def clear_cache():
    """Clear the in-memory cache."""
    _cache.clear()
    _validation_errors.clear()

```

2.2 Verify the Loader Works

STEP : Test import and loading

```

python3 -c "from track_features_loader import load_all_track_features; data = load_all_track_features();
print(f'Loaded {len(data)} circuits'); print(list(data.keys())[:5])"

```

Expected output: **Loaded 24 circuits** followed by the first 5 circuit keys. If fewer than 24 are loaded, check for missing or invalid JSON files.

PHASE 3: Modify config.py

Add new encoding maps for the track features, and add a helper function that merges circuit_config dicts with the per-circuit JSON data. The existing CIRCUITS dict, SC_PROBABILITY, VSC_PROBABILITY, and encoding maps are kept for backward compatibility.

3.1 Add New Encoding Maps

Add the following constants AFTER the existing DOWNFORCE_ENC and CONDITION_ENC (around line 400 in config.py). These maps convert human-readable labels to numeric values used by the ML model.

STEP 1: Add encoding maps after CONDITION_ENC (line ~400)

```
# -----  
# TRACK FEATURE ENCODING MAPS (Phase 5)  
# -----  
CIRCUIT_TYPE_ENC = {"street": 0, "hybrid": 1, "permanent": 2}  
DOWNFORCE_LEVEL_ENC = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5} # Already numeric 1-5  
OVERTAKE_DIFF_ENC = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5} # Already numeric 1-5  
TIRE_DEG_ENC = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5} # Already numeric 1-5  
DIRECTION_ENC = {"clockwise": 0, "counter-clockwise": 1, "figure-8": 2}  
QUALI_IMPORTANCE_ENC = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5} # Already numeric 1-5  
WEATHER_VAR_ENC = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5} # Already numeric 1-5
```

3.2 Add Helper Function

Add a function that enriches the existing circuit_config dict with data from the per-circuit JSON file. This is the bridge between old and new data sources.

STEP 2: Add get_enriched_circuit_config() function at end of config.py

```
def get_enriched_circuit_config(gp_name: str) -> dict:  
    """  
    Return circuit config enriched with per-track JSON features.  
    Falls back to the basic CIRCUITS dict if JSON is unavailable.  
    """  
    from track_features_loader import load_track_features  
  
    base_cfg = CIRCUITS.get(gp_name, {})  
    circuit_key = base_cfg.get("key", "")  
    if not circuit_key:  
        return base_cfg  
  
    track_data = load_track_features(circuit_key)  
    if track_data is None:  
        return base_cfg  
  
    # Merge: JSON data supplements (does NOT replace) base config  
    enriched = dict(base_cfg)  
    for key, value in track_data.items():  
        if key not in enriched: # Only add new keys  
            enriched[key] = value  
  
    return enriched
```

PHASE 4: Modify predictor.py - ML Feature Expansion

This is the most critical phase. Expand FEATURE_NAMES from 35 to 52 features, modify `_build_features()` to include the new track-specific features, and ensure the training pipeline generates correct historical feature vectors for the new columns.

4.1 Expand FEATURE_NAMES

Locate the FEATURE_NAMES list (currently at line ~122-172). Add 15 new track feature names and 2 derived features AFTER the existing Phase 4 features. The order MUST match the order in the np.array return at the end of `_build_features()`.

STEP 1: Add new feature names after 'fresh_tires_avail' (line ~171)

```
# Phase 5: Track-specific features (from per-circuit JSON)
"circuit_length_km", # track length in km (3.3-7.0 range)
"num_turns", # number of corners (10-27)
"circuit_altitude_m", # circuit altitude in meters (0-2240)
"longest_straight_m", # longest straight in meters (500-2200)
"track_width_m", # track width in meters (10-14)
"sm_zones", # 2026 Straight Mode activation zones (1-4)
"downforce_level", # numeric 1-5 (replaces string downforce_enc)
"overtake_difficulty", # numeric 1-5 (replaces string overtaking_enc)
"tire_degradation_track", # track tire deg level 1-5 (not driver-specific)
"sc_prob_track", # circuit-specific SC probability 0-1
"first_lap_incident_risk", # probability of T1 incident 0-1
"quali_importance", # how much quali position matters 1-5
"weather_variability", # weather change risk 1-5
"pit_time_loss_s", # pit stop time penalty in seconds (20-25)
"deg_compound_delta", # tire compound performance gap (0.1-0.5)
# Phase 5: Derived track features
"overtake_mode_efficiency", # how effective Straight Mode is at this circuit (0-1)
"track_power_sensitivity", # derived: (1/downforce) * power_unit importance
```

NOTE: N_FEATURES will automatically update since it is computed as `len(FEATURE_NAMES)`. The model cache key includes `_features_{N_FEATURES}`, so the old cached model will be invalidated and a retrain will be triggered automatically.

4.2 Modify _build_features()

In the `_build_features()` method (line ~437-691), add the new feature computations AFTER the Phase 4 fantasy features block (after the `fresh_tires_avail` computation, around line 651) and BEFORE the `return np.array([...])` statement (line ~653).

STEP 2: Add track feature computation before the return statement

```
# — Phase 5: Track-specific features from JSON —————
from track_features_loader import load_track_features
track_data = load_track_features(circuit_key) or {}

circuit_length_km = float(track_data.get("circuit_length_km", 5.0))
num_turns = int(track_data.get("num_turns", 15))
circuit_altitude_m = float(track_data.get("altitude_m", 0))
longest_straight_m = float(track_data.get("longest_straight_m", 900))
track_width_m = float(track_data.get("track_width_m", 12))
sm_zones = int(track_data.get("sm_zones", 2))
downforce_level = int(track_data.get("downforce_level", 3))
overtake_difficulty = int(track_data.get("overtaking_difficulty", 3))
tire_deg_track = int(track_data.get("tire_degradation", 3))
sc_prob_track = float(track_data.get("sc_probability", 0.40))
first_lap_risk = float(track_data.get("first_lap_incident_risk", 0.07))
quali_importance = int(track_data.get("quali_importance", 3))
weather_variability = int(track_data.get("weather_variability", 3))
pit_time_loss_s = float(track_data.get("pit_time_loss_s", 22.0))
deg_compound_delta = float(track_data.get("deg_compound_delta", 0.3))
overtake_mode_eff = float(track_data.get("overtake_mode_efficiency", 0.5))

# Derived feature: track power sensitivity
# High downforce + low power_unit importance = low power sensitivity
# Low downforce + high power_unit importance = high power sensitivity
power_enc_val = POWER_UNIT_ENC.get(circuit_cfg.get("power_unit", "medium"), 2)
track_power_sensitivity = (6 - downforce_level) * power_enc_val / 10.0
track_power_sensitivity = max(0.0, min(2.0, track_power_sensitivity))
```

4.3 Append New Features to Return Array

Modify the `np.array([...])` return statement at the end of `_build_features()` (around line 653-691). Append the 17 new values AFTER the existing Phase 4 features (`fresh_tires_avail`) and BEFORE the closing bracket.

STEP 3: Append 17 new values to the return `np.array`

```
# Phase 5: Track-specific features
float(circuit_length_km), # circuit_length_km
float(num_turns), # num_turns
float(circuit_altitude_m), # circuit_altitude_m
float(longest_straight_m), # longest_straight_m
float(track_width_m), # track_width_m
float(sm_zones), # sm_zones
float(downforce_level), # downforce_level
float(overtake_difficulty), # overtake_difficulty
float(tire_deg_track), # tire_degradation_track
float(sc_prob_track), # sc_prob_track
float(first_lap_risk), # first_lap_incident_risk
float(quali_importance), # quali_importance
float(weather_variability), # weather_variability
float(pit_time_loss_s), # pit_time_loss_s
float(deg_compound_delta), # deg_compound_delta
float(overtake_mode_eff), # overtake_mode_efficiency
float(track_power_sensitivity), # track_power_sensitivity
```

WARNING: The order of values in the `np.array` MUST exactly match the order of names in `FEATURE_NAMES`. If they are misaligned, the model will train on wrong feature-to-value mappings and produce garbage predictions.

4.4 Handle Training Data for New Features

The training loop (`train()` method, line ~764+) iterates over historical seasons and calls `_build_features()` for each driver-race pair. For historical races, the track features will be loaded from the JSON files (same data - track characteristics do not change year to year). The loader will find the JSON file by `circuit_key`, which is derived from the `CIRCUITS` dict using the GP name. No special handling is needed because the track JSON data is circuit-specific, not season-specific.

NOTE: Madrid (Madring) is a new circuit with no historical data. For training, if a historical race does not match any circuit key in the JSON files, the loader returns `None`, and the default values in the `.get()` calls will be used. This is acceptable because Madrid has round 14 in 2026, and training data only goes up to 2025.

4.5 Update `_match_circuit_cfg()` for Historical Races

Find the `_match_circuit_cfg()` helper function used in training to map historical race names to circuit configs. This function must also load track features for historical circuits. Add the enrichment call at the end of this function.

STEP 4: Enrich circuit config in `_match_circuit_cfg()`

```
# At the end of _match_circuit_cfg(), before returning the config dict:
from track_features_loader import load_track_features
circuit_key = cfg.get("key", "")
if circuit_key:
    track_data = load_track_features(circuit_key)
    if track_data:
        for k, v in track_data.items():
            if k not in cfg:
                cfg[k] = v
return cfg
```

4.6 Delete Old Model Cache

STEP 5: Force model retrain by deleting cached model

```
rm -f cache/models/ensemble_*.pkl
echo "Deleted cached model. Next run will retrain with 52 features."
```

The model cache includes the feature count in its hash key, so the old 35-feature model will automatically be considered stale. However, explicitly deleting it ensures no edge cases. The retrain will take approximately 5-8 minutes with 52 features.

PHASE 5: Modify monte_carlo.py

Integrate circuit-specific variance parameters into the Monte Carlo simulation. Currently, weather noise and SC effects use generic parameters. With per-circuit JSON data, the simulation can use more realistic variance multipliers based on track characteristics like overtaking difficulty, weather variability, and SC probability.

5.1 Add Circuit-Specific Variance Multipliers

In the `_simulate_one_race()` function (line ~112-257), add a new parameter `circuit_features` and use it to adjust the simulation physics. The function signature currently takes `sc_prob`, `vsc_prob`, `rain_risk`. Add `circuit_features` as a new parameter.

STEP 1: Modify `_simulate_one_race()` signature

Current signature (line ~112):

```
def _simulate_one_race(
    race_order, quali_order, sc_prob, vsc_prob,
    rain_risk, is_sprint, sprint_order, rng,
):
```

New signature:

```
def _simulate_one_race(
    race_order, quali_order, sc_prob, vsc_prob,
    rain_risk, is_sprint, sprint_order, rng,
    circuit_features=None, # NEW: dict from track_features_loader
):
```

STEP 2: Add circuit-specific variance logic inside `_simulate_one_race()`

After the existing weather noise block (line ~157-168), add circuit-specific adjustments:

```
# — Circuit-specific variance from track features —
if circuit_features:
    # Overtaking difficulty scales position noise
    # High difficulty (5) = Monaco: positions are stable, less shuffling
    # Low difficulty (1) = Monza: more position changes
    overtake_diff = circuit_features.get("overtaking_difficulty", 3)
    overtake_noise_factor = 1.0 + (3 - overtake_diff) * 0.15 # 0.55-1.30

    # Weather variability scales weather noise
    weather_var = circuit_features.get("weather_variability", 3)
    weather_noise_factor = 1.0 + (weather_var - 3) * 0.2 # 0.6-1.4

    # Apply overtake noise factor to weather noise
    if weather_noise > 0:
        weather_noise *= weather_noise_factor
    else:
        # Even in dry races, overtaking difficulty creates position variance
        base_overtake_noise = overtake_noise_factor - 1.0
        if base_overtake_noise > 0:
            for drv in positions:
                if drv not in dnf_set:
                    noise = rng.gauss(0, base_overtake_noise)
                    positions[drv] = max(1.0, positions[drv] + noise)

# First lap incident risk
first_lap_risk = circuit_features.get("first_lap_incident_risk", 0.07)
if rng.random() < first_lap_risk:
    # First lap incident: 1-2 cars in the mid-field are affected
    active_sorted = sorted(
        [d for d in positions if d not in dnf_set],
        key=lambda d: positions[d]
    )
    mid_field = active_sorted[5:15] # P6-P15 most affected
    if mid_field:
        n_affected = rng.randint(1, min(2, len(mid_field)))
        for drv in rng.sample(mid_field, n_affected):
            # 30% chance of DNF from first-lap incident, 70% position loss
            if rng.random() < 0.3:
                dnf_set.add(drv)
            else:
                positions[drv] += rng.uniform(1.5, 4.0)
```

```

# Straight Mode efficiency affects SC benefit
# At circuits where SM is effective, SC periods shuffle positions less
# because cars can re-overtake more easily after restart
sm_eff = circuit_features.get("overtake_mode_efficiency", 0.5)
# This is applied later in the SC benefit block
else:
    overtake_diff = 3
    overtake_noise_factor = 1.0
    weather_noise_factor = 1.0
    sm_eff = 0.5
    first_lap_risk = 0.07

```

STEP 3: Modify SC benefit block to use sm_eff

In the SC benefit block (line ~175-192), reduce the SC position gain for back-markers at circuits where Straight Mode is highly effective (cars can re-overtake after restart):

```

if sc_triggered or vsc_triggered:
    sc_lap = rng.randint(10, RACE_LAPS_TYPICAL - 10)
    sorted_drivers = sorted([d for d in positions if d not in dnf_set],
                           key=lambda d: positions[d])
    n_active = len(sorted_drivers)
    for i, drv in enumerate(sorted_drivers):
        rank = i + 1
        if rank > 5 and rank <= 15:
            # SM efficiency reduces SC benefit (easier to re-overtake)
            sc_gain_base = rng.uniform(0, 2.5) if sc_triggered else rng.uniform(0, 1.5)
            sc_gain = sc_gain_base * (1.0 - 0.3 * sm_eff) # Reduce by up to 30%
            positions[drv] = max(1.0, positions[drv] - sc_gain)
        elif rank <= 5:
            sc_loss = rng.uniform(0, 1.0)
            positions[drv] = positions[drv] + sc_loss

```

5.2 Update simulate_race_weekend()

In the simulate_race_weekend() function (line ~263-426), load track features and pass them to _simulate_one_race(). Also update the SC/VSC probability source to use the JSON data if available, falling back to config.py dicts.

STEP 4: Load track features in simulate_race_weekend()

```

# After line: circuit_key = circuit_config.get("key", "")
# Replace SC/VSC probability lookup:
from track_features_loader import load_track_features
track_data = load_track_features(circuit_key) or {}

# Use JSON-based SC/VSC probs if available, else fall back to config.py
sc_prob = track_data.get("sc_probability", SC_PROBABILITY.get(circuit_key, 0.40))
vsc_prob = track_data.get("vsc_probability", VSC_PROBABILITY.get(circuit_key, 0.30))

```

STEP 5: Pass circuit_features to _simulate_one_race()

```

# In the simulation loop, update the call to _simulate_one_race():
sim_result = _simulate_one_race(
    race_order, quali_order,
    sc_prob, vsc_prob, rain_risk,
    is_sprint, sprint_order,
    rng,
    circuit_features=track_data, # NEW parameter
)

```

PHASE 6: Modify self_improvement.py

Upgrade the bias correction system from 3 categories (street/hybrid/permanent) to per-circuit bias corrections. This allows the system to learn that, for example, Verstappen consistently outperforms predictions at Suzuka but underperforms at Baku, rather than just knowing he does well at 'permanent' circuits generally.

6.1 Add Per-Circuit Bias Tracking

In `compute_bias_corrections()` (line ~9-80), add per-circuit bias tracking alongside the existing category tracking. The `driver_errors` dict currently stores errors by `circuit_type`. Add a parallel structure keyed by `circuit_key`.

STEP 1: Add per-circuit error tracking

```
# After: driver_errors = {} (line ~24)
# Add per-circuit structure:
driver_circuit_errors = {} # driver -> circuit_key -> list of errors

# Inside the loop (line ~26-50), after line: driver_errors[driver]["overall"].append(error)
# Add:
circuit_key = entry.get("circuit_key", "")
if circuit_key:
    if driver not in driver_circuit_errors:
        driver_circuit_errors[driver] = {}
    if circuit_key not in driver_circuit_errors[driver]:
        driver_circuit_errors[driver][circuit_key] = []
    driver_circuit_errors[driver][circuit_key].append(error)
```

STEP 2: Compute per-circuit EWMA biases

```
# After computing the existing driver_biases dict (line ~53-71)
# Add per-circuit bias computation:
driver_circuit_biases = {}
for driver, circuits in driver_circuit_errors.items():
    driver_circuit_biases[driver] = {}
    for circuit_key, errors in circuits.items():
        if not errors:
            continue
        recent_errors = errors[-last_n:]
        ewma = recent_errors[0]
        for err in recent_errors[1:]:
            ewma = decay * ewma + (1.0 - decay) * err
        driver_circuit_biases[driver][circuit_key] = round(ewma, 2)
```

STEP 3: Save per-circuit biases to JSON

```
# Modify the save block (line ~74-80):
out_data = {
    "driver_biases": driver_biases,
    "driver_circuit_biases": driver_circuit_biases, # NEW
}
```

6.2 Apply Per-Circuit Biases in predictor.py

In `predictor.py`, the `predict_finishing_order()` method applies bias corrections. Find the block where `self._bias_corrections` is used and add per-circuit bias lookup.

STEP 4: Add per-circuit bias in `predict_finishing_order()`

```
# In predict_finishing_order(), after the existing category-based bias correction:
# Add per-circuit bias if available
circuit_key = self._circuit_config.get("key", "")
circuit_biases = self._bias_corrections.get("driver_circuit_biases", {})
if circuit_key and circuit_biases:
    for entry in race_predictions:
        driver = entry["driver"]
        circuit_bias = circuit_biases.get(driver, {}).get(circuit_key, 0.0)
        if circuit_bias != 0.0:
            # Per-circuit bias overrides category bias for this circuit
            entry["predicted_rank"] = max(1.0, entry["predicted_rank"] + circuit_bias)
```

6.3 Update Accuracy Logging

The accuracy log must include `circuit_key` so that per-circuit biases can be computed. In `main_logic.py`, where the accuracy log entry is created, ensure `circuit_key` is included.

STEP 5: Add `circuit_key` to accuracy log entries

```
# In main_logic.py, when creating accuracy log entries, ensure:  
# entry["circuit_key"] = circuit_cfg.get("key", "")  
# entry["circuit_type"] = circuit_cfg.get("track_type", "permanent")
```

PHASE 7: Modify main_logic.py

Update the pipeline orchestrator to use the enriched circuit config from the per-circuit JSON files. This ensures that all downstream modules (predictor, MC, chip advisor) receive the full track feature data.

7.1 Use Enriched Circuit Config

STEP 1: Replace circuit_config with enriched version

```
# In run_full_pipeline(), after line: circuit_cfg = race.get("circuit_config", {})
# Replace with:
from config import get_enriched_circuit_config
circuit_cfg = get_enriched_circuit_config(race.get("name", ""))
# If no enrichment found, fall back to basic config
if not circuit_cfg:
    circuit_cfg = race.get("circuit_config", {})
```

7.2 Pass Track Features to Monte Carlo

The circuit_cfg dict now contains all the track features from the JSON file. The simulate_race_weekend() function already receives circuit_config and will extract what it needs. No additional changes needed here since we already modified monte_carlo.py to load track features from the loader.

7.3 Update Lookahead EV Function

STEP 2: Enrich circuit config in calculate_lookahead_ev()

```
# In calculate_lookahead_ev(), after line: circuit_cfg = race.get("circuit_config", {})
# Replace with:
from config import get_enriched_circuit_config
circuit_cfg = get_enriched_circuit_config(race.get("name", ""))
if not circuit_cfg:
    circuit_cfg = race.get("circuit_config", {})
```

PHASE 8: Modify data_fetcher.py

The data_fetcher attaches circuit_config to race objects. Update the relevant function to use the enriched config so that downstream consumers get the full track features.

STEP 1: Enrich circuit_config in get_next_race() and get_race_by_round()

```
# In the functions that return race dicts with circuit_config,
# after the circuit_config dict is built from CIRCUITS, add:
from config import get_enriched_circuit_config
enriched = get_enriched_circuit_config(name) # name = GP name
if enriched:
    circuit_config = enriched
```

PHASE 9: Modify chip_advisor.py

The chip advisor uses circuit_config to score chip recommendations. With richer track features, the advisor can make more nuanced chip decisions. Update the scoring functions to use the new track feature data.

9.1 Use Per-Circuit SC Probability

STEP 1: Replace generic SC lookup with JSON-based lookup

```
# In chip_advisor.py, wherever SC_PROBABILITY is used with circuit_key,  
# prefer the per-circuit JSON data:  
from track_features_loader import get_track_feature  
  
# Replace:  
# sc_prob = SC_PROBABILITY.get(circuit_key, 0.40)  
# With:  
sc_prob = get_track_feature(circuit_key, "sc_probability",  
                             SC_PROBABILITY.get(circuit_key, 0.40))
```

9.2 Use Qualifying Importance for Chip Scoring

STEP 2: Factor quali_importance into Limitless scoring

```
# In _score_limitless(), add a bonus for high quali_importance circuits:  
quali_imp = get_track_feature(circuit_key, "quali_importance", 3)  
if quali_imp >= 4:  
    score += 1.5 # High quali importance = pole sitter gets big advantage
```

STEP 3: Factor overtake_mode_efficiency into No Negative scoring

```
# In _score_no_negative(), add SM efficiency factor:  
sm_eff = get_track_feature(circuit_key, "overtake_mode_efficiency", 0.5)  
if sm_eff >= 0.7:  
    score += 1.0 # High SM efficiency = more overtaking = more variance = NN safer
```

PHASE 10: Validation and Testing

After all code changes, run these validation steps to ensure nothing is broken and the new features are working correctly.

10.1 Schema Validation Script

STEP 1: Create `validate_track_features.py`

```
#!/usr/bin/env python3
"""Validate all track feature JSON files and the loader module."""

from track_features_loader import load_all_track_features, get_validation_errors

def main():
    print("Loading all track features...")
    data = load_all_track_features()
    print(f"Loaded {len(data)} circuits")

    # Check all 24 circuits
    expected_keys = [
        "australia", "china", "japan", "miami", "canada", "monaco",
        "spain", "austria", "britain", "belgium", "hungary", "netherlands",
        "italy", "madring", "azerbaijan", "singapore", "usa", "mexico",
        "brazil", "las_vegas", "qatar", "abu_dhabi", "bahrain", "saudi_arabia",
    ]

    missing = [k for k in expected_keys if k not in data]
    if missing:
        print(f"MISSING circuits: {missing}")
    else:
        print("All 24 circuits present")

    # Validate ranges
    for key, features in data.items():
        assert 1 <= features["downforce_level"] <= 5, f"{key}: downforce out of range"
        assert 1 <= features["overtaking_difficulty"] <= 5, f"{key}: overtake out of range"
        assert 1 <= features["tire_degradation"] <= 5, f"{key}: tire_deg out of range"
        assert 0 <= features["sc_probability"] <= 1, f"{key}: sc_prob out of range"
        assert 0 <= features["vsc_probability"] <= 1, f"{key}: vsc_prob out of range"
        assert 0 <= features["first_lap_incident_risk"] <= 1, f"{key}: fl_risk out of range"
        assert 0 <= features["overtake_mode_efficiency"] <= 1, f"{key}: sm_eff out of range"
        print(f" {key}: OK")

    errors = get_validation_errors()
    if errors:
        print(f"Validation errors: {errors}")
    else:
        print("No validation errors")

    # Check loader integration with config.py
    from config import CIRCUITS, get_enriched_circuit_config
    for gp_name in CIRCUITS:
        enriched = get_enriched_circuit_config(gp_name)
        if "downforce_level" not in enriched:
            print(f"WARN: {gp_name} not enriched with track features")
        else:
            print(f" {gp_name}: enriched OK (downforce_level={enriched['downforce_level']})")

if __name__ == "__main__":
    main()
```

STEP 2: Run the validation script

```
python3 validate_track_features.py
```

Expected: All 24 circuits loaded, all range checks pass, all GP names enriched. Fix any errors before proceeding.

10.2 Feature Count Verification

STEP 3: Verify feature count matches

```
python3 -c "from predictor import FEATURE_NAMES, N_FEATURES;
print(f'N_FEATURES = {N_FEATURES}');
print(f'Last 5 features: {FEATURE_NAMES[-5:]}');
assert N_FEATURES == 52, f'Expected 52 features, got {N_FEATURES}'"
```

Expected output: N_FEATURES = 52, last 5 features should include the new track features. If the count is wrong, check the FEATURE_NAMES list and the _build_features() return array.

10.3 Smoke Test - Build Features for One Driver

STEP 4: Test _build_features() with track data

```
python3 -c "
from predictor import F1Predictor
from track_features_loader import load_track_features

p = F1Predictor()
track_data = load_track_features('monaco')
circuit_cfg = {'key': 'monaco', 'track_type': 'street',
               'overtaking': 'very_low', 'power_unit': 'low', 'downforce': 'very_high'}
weather = {'summary_condition': 'dry', 'rain_risk': 'low'}

# Build features without full context (partial test)
p._circuit_config = circuit_cfg
p._weather = weather
p._roster = {'Verstappen': 'Red Bull'}
p._driver_standings = [{'name': 'Verstappen', 'position': 1, 'points': 100, 'wins': 5}]
p._ctor_standings = [{'constructor': 'Red Bull', 'position': 1, 'points': 200}]
p._driver_form = {'Verstappen': {'avg_position': 3, 'avg_points': 15,
                                  'dnf_rate': 0.05, 'form_score': 0.8, 'momentum_trend': 0.2,
                                  'races_counted': 10}}
p._ctor_reliability = {'Red Bull': {'dnf_rate': 0.05, 'pts_trend': 0.1}}
p._grid_penalties = {}

feats = p._build_features('Verstappen')
print(f'Feature vector length: {len(feats)}')
print(f'Expected: 52')
assert len(feats) == 52, f'Expected 52 features, got {len(feats)}'
print('SUCCESS: Feature vector has correct length')
"
```

10.4 End-to-End Pipeline Test

STEP 5: Run a full prediction pipeline

```
# This requires API access and will take 5-10 minutes for training
# Run from the project directory:
python3 -c "
from main_logic import run_full_pipeline
import json

def progress_cb(run_id, stage, status, msg, data=None):
    print(f'[{stage}] {status}: {msg}')

result = run_full_pipeline(
    run_id='test_run',
    my_drivers=['Max Verstappen', 'Lando Norris'],
    my_constructors=['Red Bull', 'McLaren'],
    budget=100.0,
    points=0.0,
    transfers=3,
    options={'race_round': 1}, # Australian GP
    progress_callback=progress_cb,
)

if result:
    race_order = result['predictions']['race_order']
    print(f'Race predictions: {len(race_order)} drivers')
    print(f'Winner: {race_order[0]["driver"]}')
else:
    print('Pipeline failed')
"
```

WARNING: The first run after feature expansion will trigger a full model retrain (~5-8 min). Subsequent runs will use the cached model. If training fails with shape mismatch errors, verify that FEATURE_NAMES length matches the _build_features() return array length.

PHASE 11: Rollback Plan

If any phase causes critical failures, follow this rollback procedure. The implementation is designed so that each phase is independently reversible.

11.1 Quick Rollback by Phase

Phase	What to Revert	How
1 (JSON files)	Remove track_features/ directory	rm -rf track_features/
2 (Loader)	Remove track_features_loader.py	rm track_features_loader.py
3 (config.py)	Remove new encoding maps + helper function	git checkout config.py
4 (predictor.py)	Remove new FEATURE_NAMES + _build_features additions	git checkout predictor.py; rm -f cache/models/ensemble_*.pkl
5 (monte_carlo.py)	Remove circuit_features parameter + variance logic	git checkout monte_carlo.py
6 (self_improvement.py)	Remove per-circuit bias tracking	git checkout self_improvement.py
7 (main_logic.py)	Remove enriched config calls	git checkout main_logic.py
8 (data_fetcher.py)	Remove enriched config calls	git checkout data_fetcher.py
9 (chip_advisor.py)	Remove JSON-based SC/SM lookups	git checkout chip_advisor.py

11.2 Nuclear Option - Full Rollback

STEP : Revert ALL changes at once

```
git checkout config.py predictor.py monte_carlo.py self_improvement.py \  
    main_logic.py data_fetcher.py chip_advisor.py  
rm -rf track_features/  
rm -f track_features_loader.py validate_track_features.py  
rm -f cache/models/ensemble_*.pkl  
echo "Full rollback complete. Run pipeline to retrain old model."
```

Implementation Order Summary

Execute these phases strictly in order. Each phase depends on the previous one. Phases 3-9 modify existing files and can be done in sequence. Phase 10 validates everything. Phase 11 is the rollback plan if anything goes wrong.

Phase	Description	Files Created/Modified	Est. Time	Risk
1	Create 24 track feature JSON files	track_features/*.json (24 files)	15 min	Low
2	Create loader module	track_features_loader.py	10 min	Low
3	Modify config.py	config.py	10 min	Low
4	Modify predictor.py (ML features)	predictor.py	30 min	High
5	Modify monte_carlo.py	monte_carlo.py	20 min	Medium
6	Modify self_improvement.py	self_improvement.py	15 min	Medium
7	Modify main_logic.py	main_logic.py	10 min	Low
8	Modify data_fetcher.py	data_fetcher.py	10 min	Low
9	Modify chip_advisor.py	chip_advisor.py	15 min	Low
10	Validation and testing	validate_track_features.py	20 min	N/A
11	Rollback plan (if needed)	N/A	10 min	N/A

Critical Warnings

- WARNING:** Phase 4 (predictor.py) is the HIGHEST RISK phase. A mismatch between FEATURE_NAMES and the _build_features() return array will produce silent model corruption - predictions will look plausible but be wrong. ALWAYS run the feature count verification (Step 10.2) after modifying predictor.py.
- WARNING:** After Phase 4, the model cache will be invalidated. The first pipeline run will take 5-8 minutes for retraining. Do NOT interrupt the training process or you will have a corrupted model cache file.
- WARNING:** The 24 JSON files MUST be created with EXACT data values from this document. Do NOT estimate or recalculate circuit features - use the values provided in Phase 1.
- NOTE:** All changes are backward compatible. If the loader cannot find a JSON file, it returns None and the code falls back to default values. This means the program will still work even if some JSON files are missing (though accuracy will be reduced).